CHANDIGARH UNIVERSITY
Discover. Learn. Empower.

PROJECT REPORT

PROCRASTINATION SPY: OPERATION DEADLINE

A C++ Based Personal Avoidance Pattern Analyzer

A Project on Object Oriented Programming in C++

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF COMPUTER APPLICATIONS (BCA)

Name	Tanisha
UID	25BCD10140
Section	25BCD-3A
Subject	Object oriented programming
Submitted to	Mr. Abhinay Sood (Assistant Professor)
Academic.Year	2025-2026

Table of Contents

1. Abstract
2. Introduction
3. Objectives
4. Literature / Existing Systems
5. System Analysis
6. System Design (Classes and Responsibilities)
7. OOP Concepts Used
8. Implementation
9. Unique Selling Points (USP)
10. Sample Input / Output
11. Testing
12. Results and Screenshots
13. Conclusion and Future Scope
14. References

1. Abstract

Students oftenknow what they are avoiding but rarely understand why. Procrastination Spy: Operation Deadline is a C++ console application built on Object Oriented Programming principles that turns the psychology of avoidance into an interactive "spy mission". Instead of tracking what students do, it analyzes what they avoid, and interrogates their excuses.

The application has three unique mechanics: an **Interrogation Room** that detects contradictions between a stated excuse and the real reason, a **Mole Hunt** that identifies the student's number one distraction and assigns a weekly mission, and a **Threat Level Countdown** that maps deadline urgency. The project demonstrates inheritance, polymorphism, encapsulation, composition and file handling while remaining lightweight and useful for student self-reflection.

2. Introduction

2.1Background

Every student procrastinates. Most productivity tools focus on doing more: timers, habit trackers and to-do lists. None of them ask the uncomfortable question: "Why did you avoid this in the first place?" Procrastination is usually not laziness. It is more often fear, overwhelm or self-deception. A tool that surfaces these patterns is genuinely valuable.

2.2 Purpose of the Project

To design a C++ application that:

- logs avoided tasks along with excuses,
- interrogates the user to uncover the real reason,
- detects contradictions between the stated and the actual reason,
- identifies the user's most frequent distraction (the "mole"), and
- assigns a deadline-driven threat level to each task.

2.3Scope

The project is a console-based, single-user application with local file storage. It intentionally avoids heavy libraries, threads and graphics so that it stays a clean demonstration of OOP design.

3. Objectives

1. Toapply OOPconcepts (inheritance, polymorphism, encapsulation, composition) in a real-life student utility.
2. To build a working C++ application that analyzes procrastination patterns.
3. To implement three unique features: Interrogation Room, Mole Hunt and Threat Level Countdown.
4. To demonstrate persistent storage using file handling.

4. Literature / Existing Systems

4.1 Existing Systems

Common student tools include Pomodoro timers, habit trackers, screen-time monitors and to-do list apps. These tools measure presence (is the student studying?) but not resistance (why are they not?).

4.2 Limitations Observed

- Productivity apps assume the problem is time management, not emotional avoidance.
- No existing C++ mini-project analyzes contradictions in a student's own excuses.
- No project pairs procrastination logging with narrative, gamified mechanics such as a spy theme.

4.3 Motivation

A tool that does not shame the user but shows them their own patterns, framed as a spy mission, is both original and psychologically meaningful.

5. System Analysis

5.1FunctionalRequirements

1. Create missions (Study / Assignment / Project) with deadlines.
2. Log avoidance sessions with duration, excuse and distraction.
3. Interrogate the user for the real reason and detect contradictions.
4. Identify the most frequent distraction (the mole).
5. Compute threat levels per mission based on deadline and avoidance count.
6. Generate a consolidated "Agent Dossier" report.
7. Save and load data from local text files.

5.2 Non-Functional Requirements

- Console UI, no graphics.
- Lightweight, with no external libraries.
- Fast and deterministic.
- Data persistence through plain text files.

5.3 Feasibility Study

Type	Assessment
Technical	Standard C++17 using fstream, vector, ctime and map
Economic	Zero cost; runs on any free IDE
Operational	Single-user; no internet required

6. System Design (Classes and Responsibilities)

Class	Type	Responsibility
Mission	Abstract base class	Stores task name, deadline, category and avoidance count; declares pure virtual getType(), virtual display(), and provides incrementAvoidance() and toFile()
StudyMission, AssignmentMission, ProjectMission	Derived classes	Each provides its own getType()
AvoidanceEntry	Concrete class	Holds one avoidance record: task name, original excuse, real reason, distraction, duration avoided, date and contradiction flag; supports toFile() / fromFile()
Interrogator	Concrete class	askRealReason(), checkContradiction(), reportContradictions() (USP 1)

Class	Type	Responsibility
MoleDetector	Concrete class	findMole(logs) and displayMoleHunt() (USP 2)
ThreatAnalyzer	Concrete class	getThreatLevel() and displayThreat() (USP 3)
Dossier	Concrete class	generateReport() aggregates everything into one report
SpyEngine	Controller class	Owns vector<Mission*> missions, vector<AvoidanceEntry> logs and the helper objects; provides addMission(), logAvoidance(), showThreatBoard(), showMoleHunt(), showDossier(), save() and load()

6.1 Class Relationships

- **Inheritance:**StudyMission, AssignmentMission and ProjectMission inherit from the abstract class Mission.
- **Composition:** SpyEngine contains its Interrogator, MoleDetector, ThreatAnalyzer and Dossier objects, and owns the collections of missions and logs.
- **Association:** MoleDetector and Dossier read the AvoidanceEntry logs; ThreatAnalyzer reads the Mission objects.

The class diagram can be drawn by hand as follows: Mission (abstract) at the top with an inheritance arrow from its three derived classes below it; AvoidanceEntry, Interrogator, MoleDetector, ThreatAnalyzer and Dossier as separate boxes; and SpyEngine at the bottom connected to all of them with composition links.

6.2 Data Flow

1. User adds a mission, which is saved to missions.txt.
2. User logs an avoidance, the Interrogation Room runs, the contradiction flag is computed and the entry is saved to logs.txt.
3. The Threat Board reads the missions and computes the threat level on demand.
4. Mole Hunt reads all logs and finds the most frequent distraction.
5. The Dossier aggregates everything into a single report.

7. OOP Concepts Used

No.	OOP concept	Where it is used	Benefit
1	Classes and objects	Mission, AvoidanceEntry, Interrogator, MoleDetector, ThreatAnalyzer, Dossier, SpyEngine	Models real entities as code
2	Encapsulation	Private data members (taskName, deadline, avoidanceCount and others) with public methods	Data is changed only through controlled functions
3	Abstraction	Abstract class Mission with pure virtual getType()	A generic mission cannot be created; only concrete types
4	Inheritance	Study, Assignment and Project missions derive from Mission	Code reuse and a clear hierarchy
5	Runtime polymorphism	display() and getType() called through Mission pointers in vector<Mission*>	One call, different behaviour per mission type
6	Composition	SpyEngine owns the analyzer objects, missions and logs	Real "has-a" relationships

No.	OOP concept	Where it is used	Benefit
7	File handling	missions.txt and logs.txt via toFile() / fromFile(), save() / load()	Data persists between runs
8	STL	vector, map (frequency counting for distractions and excuse patterns)	Less code, reliable containers

7.1 Inheritance and Abstraction

Mission is an abstractclass.It declares getType() as a pure virtual function, so only a concrete mission can be created. StudyMission, AssignmentMission and ProjectMission inherit the shared data and behaviour and supply their own type. Adding a new mission category needs only one more small derived class.

```
class Mission {

      virtual string getType() const=0;      // pure virtual
      virtual void display() const;          // virtual
      void incrementAvoidance();
      string toFile() const;
 };
```

7.2 Polymorphism

SpyEngine storesevery mission as a Mission pointer in a single vector. When the missions are displayed or saved, the same loop works for every type, and C++ picks the correct overridden function at run time.

7.3 Encapsulation

All datamembersof Mission and AvoidanceEntry are private. For example, avoidanceCount can only be increased through incrementAvoidance(), so outside code cannot set it to an invalid value.

7.4 Composition

SpyEngine is the main controller. It contains the Interrogator, MoleDetector, ThreatAnalyzer and Dossier objects, along with the missions and logs. The engine coordinates them, while each helper class has one clear job.

7.5 File Handling

Missions are written to missions.txt and avoidance records to logs.txt. Each class converts itself to a text line with toFile(), and AvoidanceEntry rebuilds itself with fromFile(). SpyEngine::save() and SpyEngine::load() restore the complete state when the program is reopened.

8.Implementation

8.1 Development Environment

Item	Detail
Language	C++17
Compiler	g++ / MinGW
IDE	Code::Blocks / VS Code
Storage	Local .txt files (missions.txt, logs.txt)

8.2 Modules

1. Missionhierarchy: the polymorphic data model.

2. AvoidanceEntry: one avoidance record.
3. Interrogator: USP 1, contradiction detection.
4. MoleDetector: USP 2, distraction frequency analysis.
5. ThreatAnalyzer: USP 3, deadline-driven threat levels.
6. Dossier: the consolidated report.
7. SpyEngine: main controller and file I/O.
8. main(): the menu-driven interface.

9. Unique Selling Points (USP)

9.1 Interrogation Room (Primary USP)

Instead of accepting a stated excuse, the application interrogates the user and then detects contradictions between the excuse and the real reason. For example, if the user says "Too tired" but then reveals "Task feels too big", the application flags a contradiction. Over time it identifies the user's most common pattern, such as "Too tired -> Task too big, 4x".

9.2MoleHunt

The application identifies the user's number one distraction (the "mole") by frequency across all logged avoidances. It assigns a weekly mission ("reduce sightings below X") and proposes a rank reward: Trainee, Junior Agent or Field Agent.

9.3 Threat Level Countdown

Each mission receives a dynamic threat level, CRITICAL, ELEVATED, GUARDED or LOW, computed from the days to the deadline and the avoidance count. This shows urgency as a clear measure instead of a vague feeling.

Feature	Ordinary productivity tool	Procrastination Spy
Focus	What the student does	What the student avoids
Excuses	Not examined	Interrogated and checked for contradictions
Distractions	Screen-time totals	Top distraction (mole) with weekly mission and rank
Deadlines	Simple reminders	Threat level from deadline and avoidance count
Engagement	Plain lists	Spy-themed, gamified experience

10. Sample Input / Output

The sessions below follow the test cases in Section 11.

Interrogation Room: the user logs 2 hours of avoidance with the excuse "Too tired". During interrogation the user admits the real reason is "Task too big". The two reasons differ, so the entry is flagged as a contradiction.

Threat Board: a mission "DBMS Assignment" with a deadline two days away appears as "DBMS - THREAT: CRITICAL".

Mole Hunt: after five logs with the distraction "Instagram", the report names Instagram as the mole with 5 sightings.

Agent Dossier: a full report showing the top excuse, the contradiction count, the mole and a final verdict.

11. Testing

Test ID	Input	Expected result	Status
T1	Add mission "DBMS Assignment", deadline 2 days away	Mission saved	Pass
T2	Log avoidance: 2 hrs, excuse "Too tired", real reason "Task too big"	Contradiction flagged	Pass
T3	Log avoidance: 1 hr, excuse "Distracted", real reason "Distracted"	No contradiction	Pass
T4	View Threat Board	Shows "DBMS - THREAT: CRITICAL"	Pass
T5	View Mole Hunt after 5 logs with distraction "Instagram"	Mole = "Instagram", 5 sightings	Pass
T6	View Agent Dossier	Full report with top excuse, contradiction count, mole and verdict	Pass
T7	Save and reopen	All missions and logs restored	Pass

13. Conclusion and Future Scope

Conclusion

Procrastination Spy successfully demonstrates OOP principles in a genuinely novel context. Unlike standard student mini-projects that only track data, this application interprets data: it detects contradictions, identifies distraction patterns and quantifies deadline urgency. The spy framing adds engagement without heavy code, making the project both fun to demo and meaningful to use.

Future Scope

1. A GUI version using Qt.
2. Weekly email-style "intelligence briefings".
3. Multi-user agent profiles.
4. Export of the dossier as a PDF.

14. References

1. Bjarne Stroustrup, The C++ Programming Language, 4th Edition.
2. Herbert Schildt, C++: The Complete Reference.
3. cppreference.com, C++ language and standard library reference.
4. Pychyl, T. A., Solving the Procrastination Puzzle (psychology reference for avoidance patterns).